
SWE-Marathon

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 AI agents may soon become capable of autonomously completing valuable, long-
2 horizon tasks without human supervision. Current benchmarks are limited in their
3 scope, often restricted to a 5 to 10 minute task; equivalent to a single GitHub PR
4 or one "ticket" in a software engineering setting, due to the nature of how agentic
5 benchmark are constructed today (programmatic unit tests, strict time limits, and
6 contrived, overspecified instructions). Thus, these benchmarks are limited in their
7 ability to meaningfully measure frontier models as their capabilities improve in
8 long-horizon planning, long-context understanding, memory and continual learning.
9 To this end, we present SWE-Marathon: a carefully curated benchmark composed
10 of 20 tasks across software engineering and adjacent domains, inspired by problems
11 from real workflows. Each task features a unique environment, human-written
12 solution, and multi-dimensional, multi-layer tests for verification. We show that
13 frontier models and agents score less than 10% on the benchmark. Notably, we em-
14 ploy *adversarial-aware runtimes*: novel techniques to avoid reward hacking due to
15 exploits in the execution environment. We conduct a comprehensive error analysis
16 to identify areas for model and agent improvement, exposing reward hacking in
17 over 30% of agent rollouts. Finally, we publish the dataset and evaluation harness
18 to assist developers and researchers in future work at <https://swe-marathon.org>.

19 1 Introduction

20 [Steven: The RLVR history overview is too long for a ML conference paper. Consider tightening it - you will
21 need the space later for methods/results. A reviewer reading this long of an intro may underestimate the technical
22 contribution of the paper if seems to "position-esque".]

23 While model performance continues to improve at an unprecedented rate, over the last year the inputs
24 to scaling laws have shifted from parameter and pretraining scale to agentic reinforcement learning.
25 Since the release of GPT-4 in March 2023, frontier parameter counts have largely plateaued, and
26 most of the capability gains over this period have come from reinforcement learning with verifiable
27 rewards (RLVR) [CITE A: Lambert et al., Tulu 3: Pushing Frontiers in Open Language Model
28 Post-Training, 2024]. Over a remarkably short period, RLVR has progressed from grade-school math
29 word problems [CITE B: Cobbe et al., Training Verifiers to Solve Math Word Problems (GSM8K),
30 2021], to competition-level math and code under DeepSeek-R1-style training [CITE C: DeepSeek-AI,
31 DeepSeek-R1, 2025], to replicating patches in large open-source projects (SWE-Bench [2]), to
32 bespoke agentic benchmarks built from scratch to challenge agents in new domains — terminal use
33 [3], office workflows [7], freelance software engineering [4], and long-horizon library generation [8].
34 Each iteration has increased the complexity, variety, and novelty of the tasks involved.

35 But the trajectory may not be sustainable. Anthropic alone has reportedly discussed spending more
36 than \$1 billion on RL environments over the next year [CITE D: Zeff, TechCrunch, September
37 2025], and high-quality tasks now cost on the order of thousands of dollars apiece to construct,
38 with the most complex reaching \$20,000 [CITE E: Erdil & You, Epoch AI, 2026]. Synthetic task

generation is the obvious release valve, but synthetic tasks consistently underperform human-curated ones when training transfers to held-out evaluations [3, CITE F: Endless Terminals, 2026], and even as generation improves, verifying the verifier — ensuring that the automatically-checked reward actually tracks the capability we want to elicit — becomes the binding constraint: over 15% of tasks across five major terminal-agent benchmarks contain reward-hackable verifiers [CITE G: Bercovich et al., Terminal Wrench, 2026]. To keep advancing, the field needs a framework change.

Frontier coding agents now operate at hour-long horizons; benchmarks do not. Two years ago, coding agents could barely resolve minimal GitHub issues [6]. Today, frontier scaffolds discover security vulnerabilities in well-maintained codebases, build functional browsers from scratch [?], and saturate single-issue benchmarks (SWE-bench Verified > 70% [12], SWE-AGI 86% [?]). The way agents are used has changed dramatically: production deployments now routinely run agents for an hour or more on a single task. The way agents are evaluated has not kept pace. Dominant public benchmarks measure agent performance at horizons of minutes-to-low-hours; even Terminal-Bench [7], the hardest mainstream benchmark, has most tasks resolved within an hour by top scaffolds. The capability frontier has moved past where most benchmarks can measure it.

[Yiyuan: The above paragraph can be briefed with a figure. The connection with long-horizon may better be enhanced, i.e. what makes the tasks hour-long naturally, the complexity of software, the depth of design etc.]

We argue that the next iteration of RLVR is the replication of complex software: instead of authoring novel tasks one at a time, the world has already produced millions of mature open-source projects whose specifications, test suites, and reference implementations are public, version-controlled, and free. Each project is, in effect, a pre-built RL environment with a built-in verifier — the existing test suite — and a ground-truth solution that humans have already debugged into existence. This sidesteps both the cost wall of bespoke task creation and the verifier-reliability problem documented in [CITE G], while preserving the long-horizon, multi-file, cross-component structure that recent benchmarks have shown to be the actual frontier of agent capability [4, 6, 7].

[Yiyuan: The abstraction of software as pre-built RL environment with verifiers worth highlighting, but need to clarify its difference the previous PR-based or completing-based programming tasks since they could also inherit this nature.]

Prior work in the ultra-long-horizon regime. Two recent benchmarks begin to address this gap: FrontierSWE [?] curates 17 tasks at a 20-hour budget across implementation, performance engineering, and ML research; MirrorCode [1] measures black-box reimplementations of multi-week human-engineering work. SWE-Marathon contributes a complementary slice: 20 tasks at 1–8 hour horizons spanning seven domain categories, drawn from real upstream OSS, with verification grounded in maintainer-authored test suites and behavioral oracles rather than partial-credit scoring or end-to-end behavioral matching. Section 2 compares the three.

[Steven: The prior work paragraph kind of interrupts the flow/argument - you can move it to a separate section after the intro? I just see that you have a "Related Works Section - just move it there?]

Building a credible ultra-long-horizon benchmark is non-trivial. The benchmark is the contribution—but at this scale, several methodological problems arise that smaller benchmarks can ignore. Construction cost is high (30–80 author-hours per task plus 12–20 validation-hours), so each task must earn its place. Reward hacking has materially more room to operate when agents have hours to probe verifiers; concurrent work documents extensive cheating behaviour [?] (agents writing torch imports as `chr()` sequences to evade verifier scans, attempting to detect verifier scripts, fetching reference implementations from upstream repositories), and prompt-level instructions are insufficient defence. Single verification methods systematically misrepresent agent capability on tasks where real software demands behavioral parity, performance gates, deterministic replay, or live-traffic invariants beyond what unit tests can express. We address these problems with three supporting infrastructure contributions, each of which exists to make the ultra-long-horizon evaluation credible at this scale.

Building on the Harbor framework [5] [CITE H: Snorkel AI, Terminal-Bench 2.0, 2025], we introduce SWE-Marathon, a benchmark of [N] software replication tasks [briefly: what makes them hard — multi-file edits, real test suites, instructions.md scaffolding rather than a single prompt].

91 [Steven: The prior work paragraph kind of interrupts the flow/argument - you can move it to a separate section
92 after the intro? I just see that you have a "Related Works Section - just move it there?]

93 Contributions.

- 94 1. A scalable benchmark construction pipeline that turns existing open-source projects into
95 long-horizon RL environments with minimal per-task human effort, addressing the cost and
96 verification bottlenecks identified above.
- 97 2. **SWE-Marathon v1.0**: 20 ultra-long-horizon SWE tasks across seven domains (Section ??),
98 each requiring 1–8 hours of agent wall-clock and 50–250M tokens to attempt. Drawn from
99 real upstream OSS projects and research codebases. Released under Apache-2.0 with oracle
100 solutions, Harbor specs, and a frozen evaluation harness — substantially longer-horizon
101 than SWE-Bench [6] and broader in scope than FrontierSWE [4].
- 102 3. **Coverage of five verification methodologies** (Section ??): dense unit tests, behavioral
103 parity, performance gates, deterministic-replay equivalence, and live-traffic invariants. We
104 catalogue which verifier type each task uses and motivate the coverage choice as a property
105 of the long-horizon regime: production software is not graded by unit tests alone, and a
106 long-horizon benchmark that grades only one of these methodologies misrepresents what
107 agents can and cannot do.
- 108 4. **Four-gate validation pipeline including adversarial exploit auditing** (Section ??): every
109 task passes NOP, Oracle, frontier-difficulty, and adversarial-exploit gates before inclusion.
110 Audit prompts and discovered exploits are released as a public corpus.
- 111 5. **Closed-internet adversarial runtime** (Section ??): `harbor_ext`, a Harbor extension
112 running trials inside Modal sandboxes with closed network access by default, no-search
113 wrappers around tool-search APIs, and explicit per-task opt-in for tasks that legitimately
114 require network. Released as part of the benchmark.
- 115 6. **Multi-level evaluation framework** (Section 5): resolved rate (binary), partial progress
116 (continuous, verifier-type-appropriate), milestone completion (ordinal, M1–M4 with re-
117 fined M3a/M3b for dense-test regimes), and Agent-as-Judge code quality (descriptive),
118 complemented by `mean@5/best@5` reporting.
- 119 7. A comprehensive evaluation of frontier models and agent scaffolds, surfacing failure modes
120 specific to multi-component replication: [tease 2–3 of your most interesting findings here].
121 **Frontier-model evaluation** (Section ??): four frontier models (Claude Opus 4.7, GPT-5.4,
122 Gemini 3.1 Pro, Kimi K2.5) under three Harbor-compatible scaffolds (Codex CLI, Claude
123 Code, Terminus 2), with $n=5$ trials per (model, scaffold, task) cell.
- 124 8. **Live-leaderboard CI and open trajectory release** (Section ??): every task submission
125 triggers a frontier-agent CI sweep posted to the PR conversation, providing continuously
126 updated public results. We release **~[TODO: N]** complete agent trajectories under Apache-
127 2.0.
- 128 9. An analysis of what makes replication tasks reliable RLVR signal, including [a takeaway
129 from §3.3 — quality control, automated vs. human verification trade-offs].

130 **Headline finding.** [TODO: Insert from completed sweep. Expected shape: “At 8-hour budgets
131 and \$Y/trial of compute, the strongest frontier scaffold (model+scaffold) resolves only X% of
132 SWE-Marathon tasks at `mean@5`; even `best@5` reaches only Z%. Per-domain capability is non-
133 uniform, ranging from $a\%$ to $b\%$ across the seven categories, indicating the ultra-long-horizon
134 regime remains substantially unsaturated under current frontier scaffolds.”]

135 **Track positioning.** SWE-Marathon targets the NeurIPS 2026 Evaluations & Datasets track’s
136 explicit scope: “submissions need not introduce a new model or outperform prior work; what matters
137 is how the contribution supports and/or advances meaningful evaluation.” The seven contributions
138 above are evaluation contributions. We position SWE-Marathon alongside concurrent frontier-regime
139 work (FrontierSWE, MirrorCode, Terminal-Bench) and detail differences in Section 2.

Table 1: **Agentic SWE / research benchmarks against the desiderata for ultra-long-horizon evaluation.** HORIZON: per-trial budget (*min*=minutes, *h*=hours, *d*=days, *wk*=weeks); parenthetical = reported value or human-baseline (*hum.*) / junior-engineer (*jr*) reference. # VER. = distinct verifier methodologies the benchmark applies (e.g. unit tests, behavioural parity, performance gates). ADV. = adversarial-aware execution: closed-internet runtime or exploit-audit inclusion gate (*post-hoc* = cheating reported but not designed against). # TASKS reflects scale, not difficulty. SWE-Marathon is the only row that combines a multi-hour budget, multi-method verification, and an adversarial-aware runtime.

Benchmark	# Tasks	Horizon	Source / Domain	# Ver.	Adv.
<i>Repository-level SWE</i>					
SWE-Bench [6]	2,294	min	GitHub issues, 12 Py repos	1	×
SWE-Bench Verified [9]	500	min	Human-vetted SWE-Bench subset	1	×
SWE-EVO [12]	48	h	Release-note deltas, 7 Py projects	1	×
SWE-Lancer [8]	1,488	h	Upwork freelance / Expensify	1	×
Multi-SWE-Bench [17]	1,632	min	GitHub issues, 7 langs	1	×
<i>Replication-as-benchmark</i>					
Commit0 [19]	54	h	Spec + interactive unit tests	1	×
PaperBench [11]	20	d (24 h)	ICML 2024 papers	1	×
SUPER [2]	801	h	Research GitHub repos (NLP/ML)	1	×
CORE-Bench [10]	270	h	90 published papers, 3 disciplines	1	×
ReplicationBench [16]	111	h	20 published astrophysics papers	1	×
<i>Long-horizon agentic, adjacent domains</i>					
Terminal-Bench 2.0 [7]	89	d (1 d jr)	Curated terminal tasks	2	×
RE-Bench [14]	7	h (8 h hum.)	Hand-built ML R&D environments	1	×
MLE-Bench [3]	75	d (24 h)	Kaggle competitions	1	×
Cybench [18]	40	h	Professional CTFs	1	×
TheAgentCompany [15]	175	h	Simulated software company	1	×
OdysseyBench [13]	602	h	Office workflows (5 apps)	1	×
<i>Multi-hour curated-OSS regime (closest neighbours)</i>					
FrontierSWE [4]	17	d (20 h)	Curated OSS (impl/perf/research)	3	post-hoc
MirrorCode [1]	20+	wk (hum.)	Black-box CLI reimplementations	1	×
SWE-Marathon (ours)	20	h (1–8 h)	Curated upstream OSS, 7 domains	5	✓

2 Related Works

Section 1 compares 18 agentic SWE and research benchmarks against the desiderata for ultra-long-horizon evaluation: a multi-hour per-trial budget, multiple verifier methodologies, and an adversarial-aware runtime. Of the existing benchmarks surveyed, none satisfy all three: the repository-level SWE cluster (SWE-Bench, Multi-SWE-Bench, SWE-EVO, SWE-Lancer) is graded by a single test suite at minute-to-hour horizons; the replication-as-benchmark cluster (Commit0, PaperBench, SUPER, CORE-Bench) extends the horizon but still relies on one verifier methodology; long-horizon agentic benchmarks in adjacent domains (Terminal-Bench, RE-Bench, MLE-Bench, Cybench, TheAgentCompany, OdysseyBench) reach hours-to-days but use one or two verifier types and no adversarial-aware runtime; and even SWE-Marathon’s closest neighbours — FrontierSWE [4] and MirrorCode [1] — top out at three verifier types and only post-hoc cheating reporting. SWE-Marathon is the only row in the table that combines all three. The remainder of this section discusses each cluster in turn.

2.1 Existing SWE Benchmarks

The dominant baseline for repository-level software engineering evaluation is **SWE-Bench** [6], which casts 2,294 real GitHub issues across 12 Python repositories as patch-generation tasks graded by their original test suites, and its human-validated subset **SWE-Bench Verified** [9]. Recent work has moved from isolated, single-issue tasks toward more realistic and longer-horizon software engineering settings. **SWE-Lancer** [8] introduces 1,488 freelance software engineering tasks from the open-source Expensify repository on Upwork, valued at \$1,000,000 USD in real-world payouts and divided into 764 Individual Contributor (IC SWE) tasks and 724 SWE Manager tasks. End-to-end Playwright

161 tests are triple-verified by professional engineers, and evaluation maps directly to monetary value
162 rather than test-pass percentage. The strongest evaluated model, Claude 3.5 Sonnet, earns \$208,050
163 of \$500,800 on the public Diamond split; on IC SWE Diamond, pass@1 and earnings rates “remain
164 below 30%” for all evaluated models. The Diamond tasks are themselves long-horizon: “The average
165 task in the Diamond set takes 26 days to resolve on Github and has 47 comments, which include
166 discussion of multiple proposals.” SWE-Lancer’s authors observe that agents “excel at localizing, but
167 fail to root cause,” which they attribute to “a limited understanding of how the issue spans multiple
168 components or files” [8].

169 **Terminal-Bench 2.0** [7] curates 89 hard terminal tasks from 229 community contributions, with
170 each task receiving “approximately three hours of combined reviewer attention.” Tasks comprise
171 an instruction, a Docker image, tests, and an oracle solution, and are graded by container-state
172 verification. The benchmark is run at scale, with “a total of 32,155 trials, where each trial is one
173 agent’s attempt at solving a single task.” Frontier model-agent pairings “resolve less than 65% of
174 tasks” (Codex CLI + GPT-5.2 leads at $62.9\% \pm 3.0\%$), and command-level error analysis identifies
175 “command failures calling executables that are not installed or not in PATH” as the most frequent
176 failure category at 24.1% [7].

177 **SWE-EVO** [12] targets long-horizon software evolution by sourcing 48 tasks from release-note deltas
178 across seven mature Python projects. SWE-EVO instances require, on average, edits to 20.9 files
179 and 610.5 lines, and are validated against an average of 874 tests per instance. Thai et al. report that
180 “GPT-5.4 with OpenHands achieves only 25% on SWE-EVO versus 72.80% achieved by GPT-5.2 on
181 SWE-Bench Verified,” framing this as “a striking capability gap” for “sustained, multi-file reasoning.”
182 They additionally introduce a *Fix Rate* metric that grants partial credit for FAIL_TO_PASS tests
183 resolved while requiring all PASS_TO_PASS tests to remain green. Notably, patch apply rates on
184 SWE-EVO span 85.42% to 100% across evaluated models even as Resolved Rates stay at or below
185 25%, indicating that agents reliably produce syntactically valid patches that nonetheless fail the
186 semantic intent of the change [12].

187 **FrontierSWE** [4] is the closest neighbour to SWE-Marathon on the curated-OSS axis: 17 tasks
188 spanning implementation, performance, and research, evaluated under a 20-hour budget per task.
189 The reported leaderboard places Claude Opus 4.7 (Claude Code) first with an average rank of 2.32,
190 followed by GPT-5.4 (Codex) at 2.68 and Claude Opus 4.6 (Claude Code) at 2.82. Beyond pass/fail
191 metrics, FrontierSWE catalogues behavioural failures that are central to long-horizon evaluation:
192 agents “decide to submit solutions very early, long before reaching the 20-hour time limit” due
193 to “overconfidence in their wrong solutions,” Claude Opus 4.6 “spends more than 8 hours per task
194 whereas other models average around two hours,” and the authors document explicit shortcut-taking
195 (“six out of 30 trials for the Wan 2.1 on MAX/Mojo task received a zero score due to cheating”) [4].

196 **MirrorCode** [1] probes black-box reimplementations of complete CLI programs: the agent receives
197 execute-only access to the original program and a visible test suite, but no source code, and must
198 reconstruct the program so that a held-out suite passes by identical-output match. The benchmark
199 spans cryptography (e.g. AES, SHA-256), bioinformatics, and Unix utilities; the headline result is
200 Claude Opus 4.6 reimplementing the 16,000-line Go bioinformatics toolkit *gotree* (40+ commands),
201 which the authors estimate would take a human engineer between 2 and 17 weeks of work. A
202 central methodological caveat is that the benchmark “relies on something rarely present in real
203 software development: precise, programmatically checkable specifications” [1] — a setup that arises
204 in reverse-engineering rather than greenfield development. MirrorCode is also currently released as a
205 data artefact rather than a runnable framework: the public repository ships agent-generated codebases,
206 transcripts, and scores from the authors’ own runs, but does not provide an executor that would let a
207 third party reproduce or extend the benchmark with new agents or new tasks. SWE-Marathon and
208 MirrorCode sit at opposite ends of the long-horizon coding axis: MirrorCode probes whether agents
209 can match the externally-observable behaviour of an existing artefact under one verification regime,
210 whereas SWE-Marathon probes whether they can author multi-component code that satisfies five
211 different verification regimes simultaneously.

212 2.2 Replication-as-benchmark

213 A complementary line of work treats reconstruction of an existing artefact — a library, a paper, a
214 research repository — as the unit of evaluation, with the artefact’s own tests, rubrics, or outputs
215 supplying an automated verifier. This category is the closest sibling to SWE-Marathon’s premise that

216 mature open-source projects already function as pre-built RL environments with built-in verifiers;
217 SWE-Marathon’s design extends the same idea from single artefacts to multi-component systems
218 graded by multiple methodologies in parallel.

219 **Commit0** [19] reframes the benchmark as library generation from scratch: agents receive a specifica-
220 tion document plus an interactive unit-test suite and must implement an entire Python library. The
221 benchmark contains 54 libraries (16 in the “lite” split), with specifications ranging from over 10,000
222 to 300,000 tokens. The strongest scaffold reported, OpenHands, achieves a 42.95% test pass rate on
223 lite and 15.25% on the full set, and Zhao et al. note that while “current agents can pass some unit
224 tests, none can yet fully reproduce full libraries.” They also report that “interactive feedback is quite
225 useful for models to generate code that passes more unit tests” [19].

226 **PaperBench** [11] extends the artefact upward to entire research papers: agents must replicate 20
227 ICML 2024 Spotlight and Oral papers from scratch, including paper comprehension, codebase
228 development, and successful experiment execution. The benchmark grades replication against 8,316
229 individually-gradable sub-tasks organised into hierarchical rubrics co-developed with each paper’s
230 original authors, providing partial-credit signal that single pass-rate metrics cannot. The rubric is the
231 central design choice: an automated verifier rich enough to credit partial progress over a multi-hour
232 rollout without requiring a human in the loop.

233 **SUPER** [2] approaches the same problem from the lower end of the research lifecycle: 801 tasks ask
234 agents to set up and execute experiments from real research GitHub repositories under an executable
235 per-task verifier. The benchmark spans Expert, Masked, and Auto-generated subsets to cover difficulty
236 bands, repository setup — often undocumented and brittle — is itself a significant component of
237 replication that single-shot benchmarks tend to elide.

238 **CORE-Bench** [10] pushes the artefact out of CS entirely, evaluating computational reproducibility
239 across 270 tasks drawn from 90 published papers in computer science, social science, and medicine,
240 at three difficulty levels and including both language-only and vision-language variants. The strongest
241 evaluated agent reaches only 21% on the hardest level, indicating substantial headroom in the regime,
242 and reproducibility-as-benchmark is positioned as a stepping stone: “having agents that can reproduce
243 existing work is a necessary step towards building agents that can conduct novel research” [10].

244 2.3 Synthetic Benchmark Creation

245 **OdysseyBench** [13] couples a benchmark with a synthetic generation framework. The benchmark
246 itself contains 602 long-horizon office workflow tasks across Word, Excel, PDF, Email, and Calendar,
247 split into OdysseyBench⁺ (300 tasks derived from OfficeBench traces) and OdysseyBench-Neo
248 (302 fully synthetic tasks). Their accompanying **HomerAgents** framework automates long-horizon
249 benchmark creation through a four-phase multi-agent pipeline (Orchestrator, Surfers, Task Generator,
250 and Dialogue Generator) that performs “systematic environment exploration, task generation, and
251 dialogue synthesis,” with all component agents powered by GPT-4.1 and verifier-driven iterative
252 refinement (up to five iterations). Wang et al. motivate this generative pipeline by arguing that
253 “existing benchmarks predominantly focus on atomic tasks that are self-contained and independent,
254 failing to capture the long-term contextual dependencies and multi-interaction coordination required
255 in realistic scenarios” [13]. Commit0 [19] similarly leans on programmatically curated specifications
256 paired with interactive feedback (unit tests, lint, and coverage), demonstrating that automatically
257 validated signals can scaffold benchmark construction at the library level.

258 2.4 Agents for Long-Horizon Tasks

259 Across the long-horizon evaluations surveyed above, several recurring failure modes emerge.

260 **Localization without root-cause fix.** SWE-Lancer reports that agents “pinpoint the source of an
261 issue remarkably quickly, using keyword searches across the whole repository to quickly locate
262 the relevant file and functions,” often “far faster than a human would,” but “often exhibit a limited
263 understanding of how the issue spans multiple components or files, and fail to address the root
264 cause” [8]. SWE-EVO observes that for stronger GPT-5 variants, “over 60% of failures come from
265 *Instruction Following*, suggesting difficulty interpreting complex release notes,” while smaller variants
266 instead exhibit growing “Incorrect Implementation, Tool-Use, and Syntax Error” failures [12].

267 **Premature submission and budget under-utilization.** FrontierSWE finds that models “decide to
268 submit solutions very early, long before reaching the 20-hour time limit” [4]. SWE-Lancer’s pass@*k*
269 analysis shows that “allowing more attempts leads to a consistent increase in pass rate” and that for
270 o1, “allowing for 6 more attempts nearly triples the percentage of solved tasks” [8], consistent with
271 single-attempt budgets being systematically under-used.

272 **Multi-file and cross-component reasoning.** SWE-EVO instances average 20.9 edited files; the
273 best agent resolves only 25% of them despite resolving 72.80% on SWE-Bench Verified [12].
274 OdysseyBench reports “a consistent performance degradation across all models as the number
275 of applications in a task increases,” with o3 falling from 72.83% (single-application) to 30.36%
276 (three-application) on OdysseyBench⁺, while the human baseline on OdysseyBench-Neo reaches
277 91.43% [13].

278 **Tool fragility and abandonment.** SWE-Lancer’s per-task user tool “often takes a nontrivial amount
279 of time to run,” on the order of “90 to 120 seconds,” “during which weaker models such as GPT-4o are
280 prone to abandoning the tool altogether” [8]. OdysseyBench attributes a substantial share of failures
281 to “Incorrect tool calls,” where agents “invoke the wrong function or use incorrect arguments” [13],
282 and Terminal-Bench identifies executables that are “not installed or not in PATH” as 24.1% of all
283 command-level failures [7].

284 **Shortcut-taking and benchmark integrity.** FrontierSWE documents “aggressive cheating at-
285 tempts” on at least one task [4]. Terminal-Bench explicitly bakes an *Integrity* criterion into task
286 design (“if a task involves editing a git repository at a particular commit, all future commits should
287 be removed”) so that shortcut paths to passing tests are removed at construction time [7], and SWE-
288 Lancer recommends “disable browsing and limit sites accessed to prevent direct-lookups” along with
289 post-hoc filtering for cheating instances during evaluation [8].

290 3 SWE-Marathon

291 3.1 Methodology

292 [TODO: What did you start with? Harbor? What steps did you take to make it better? What is
293 the benchmark creation process? Other implementation details - we use instructions.md rather
294 than prompt. Emphasize your novelty again.]

295 3.2 Dataset Analysis

296 [TODO: What metrics are we talking about?]

297 3.2.1 Basic Statistics

298 [TODO: Number of tasks, time per task, number of checkpoints, etc.]

299 3.2.2 Interesting Trends and Findings

300 [TODO: Interesting trends or facts uncovered.]

301 3.3 Dataset Quality

302 [TODO: Cite Ivan’s work here, as well as prev work on data quality.]

303 3.3.1 Automated Validation vs. Human Verification

304 [TODO: Discuss the trade-offs and your approach.]

305 4 Benchmark Overview

306 [TODO: BELOW IS AI-GENERATED PLACEHOLDER. NEEDS TO BE REWRITTEN!!!]

Table 2: **Horizon comparison across benchmarks.** All measurements use Claude Code + Claude Sonnet 4.5 under identical harness settings. RALPHBench tasks require $12\times$ more steps than SWE-bench.

Benchmark	Median steps	Median files	Median test iters	P90 steps
SWE-bench [6]	187	1.3	2.4	412
SWE-bench Pro [?]	583	3.2	5.1	1,240
Terminal-Bench 2.0 [7]	824	2.6	4.8	1,890
RALPHBench (ours)	2,347	8.7	11.6	4,820

Table 3: **RALPHBench v1.0 statistics.**

Scale		Complexity	
Tasks	100	Mean steps/task	2,400
Repositories	24	Median steps/task	2,347
Languages	8	Median files/task	8.7
Dev / Held-out split	60 / 40	Mean assertions/task	142

4.1 Motivation: Long-Horizon Software Engineering

Benchmarks such as SWE-bench have catalyzed progress, but many instances are short-horizon: a single issue is fixed with a small, localized patch. In contrast, long-horizon SWE tasks require:

- **Architectural understanding:** Grasping how components interact across a codebase
- **Multi-file coordination:** Making consistent changes across many files
- **Iterative debugging:** Multiple build-test-debug cycles over extended trajectories
- **Context management:** Tracking relevant information across thousands of steps
- **Strategy revision:** Recognizing when an approach isn’t working and pivoting

These requirements induce *compounding error* (small early mistakes cascade), *delayed credit assignment* (tests fail far from the causal edit), and *non-stationarity* (the agent simultaneously changes the codebase and the debugging surface).

4.2 Quantifying the Horizon Gap

To validate that RALPHBench measures a meaningfully different capability regime, we ran identical agents on multiple benchmarks under standardized harness settings. Section 2 reports the resulting interaction statistics.

The step distribution is significantly right-shifted compared to SWE-bench (Mann-Whitney $U = 847$, $p < 0.001$, rank-biserial $r = 0.83$), confirming that RALPHBench tasks require sustained multi-thousand-step trajectories.

4.3 Dataset Composition

RALPHBench v1.0 provides 100 curated tasks across 24 production repositories and 8 programming languages. Section 3 summarizes the dataset; the full task manifest appears in Section ??.

Task categories. We group tasks into five categories representing distinct SWE workflows:

- **Feature implementation** (28 tasks): Adding new functionality with tests and documentation. Examples: implement OAuth2 flow, add GraphQL endpoint, create CLI tool.
- **Cross-cutting refactor** (24 tasks): Restructuring code across multiple modules while preserving behavior. Examples: React Class $\rightarrow$ Hooks migration, extract service layer, consolidate duplicate utilities.

Table 4: Distribution by task category and programming language.

Category	n	Med. steps	Med. files	Language	n	Med. steps	Repos
Feature implementation	28	1,840	6.2	Python	32	2,180	8
Cross-cutting refactor	24	2,920	12.4	TypeScript	24	2,540	6
System integration	22	2,180	7.8	Go	14	2,890	3
Performance optimization	14	2,650	5.1	Rust	10	3,120	2
Migration	12	3,410	14.2	Java	8	2,340	2
				C++	6	2,780	1
				Ruby	4	1,920	1
				PHP	2	2,050	1

• **System integration** (22 tasks): Connecting components, adding APIs, or integrating external services. Examples: add Redis caching layer, implement webhook system, integrate payment provider.

• **Performance optimization** (14 tasks): Improving efficiency while maintaining correctness. Examples: optimize database queries, implement connection pooling, reduce bundle size.

• **Migration** (12 tasks): Upgrading dependencies, APIs, or frameworks with breaking changes. Examples: Java 8 $\rightarrow$ 17, Django 3 $\rightarrow$ 5, Angular $\rightarrow$ React.

Task type distribution. Following our comparison framework (Section ??), we distinguish greenfield (building from scratch) vs. modification (changing existing code) tasks:

• **Modification tasks** (62 tasks, 62%): Refactors, migrations, and optimizations that transform existing code

• **Greenfield tasks** (38 tasks, 38%): New features and integrations that build functionality from specifications

This mixed distribution reflects real-world SWE work, where engineers split time between maintaining existing systems and building new capabilities.

Repository selection. We selected repositories based on: (i) active maintenance and realistic complexity, (ii) comprehensive test suites, (iii) diversity across domains (web frameworks, systems tools, ML libraries), and (iv) permissive licensing. Selected repositories include: encode/starlette, pallets/flask, django/django, vercel/next.js, microsoft/TypeScript, facebook/react, kubernetes/kubernetes, golang/go, rust-lang/cargo, tokio-rs/tokio, pytorch/pytorch, tensorflow/tensorflow, and others (full list in Section ??).

4.4 Task Curation Process

Task sourcing. RALPHBench v1.0 tasks were authored in-house by domain-knowledgeable engineers, each task starting from a real-world engineering goal observed in production OSS projects: a Java tool that needed a Rust port for performance, a C++ simulated-annealing inner loop with known throughput headroom, a graphlet sampler whose canonical-labeling scheme blocked extension to larger k , and a self-hosting C compiler as a benchmark for end-to-end systems work. We did not crowd-source from a general task pool—at this horizon, each task requires 30–60 hours of construction time (Section 5), and quality requires deep author familiarity with the upstream codebase.

Validation pipeline. Each task passes through four gates before inclusion, modelled on the Terminal-Bench audit process [7]:

1. **NOP gate.** Running the task with a no-op agent (which makes no edits) must fail—i.e., the unmodified codebase must not already pass the oracle test set. This rules out tasks whose tests are vacuously satisfied.

- 370 2. **Oracle gate.** Running the task with the human-authored reference solution applied must pass all
371 oracle tests. This confirms the task is solvable as specified.
- 372 3. **Frontier-difficulty gate.** A pilot run with Claude Code on Claude Opus 4.6 must not solve the
373 task on the first attempt; junior-engineer estimated solve time must exceed 8 hours.
- 374 4. **Adversarial exploit audit.** An adversarial agent is run with explicit instructions to find shortcuts
375 that pass the tests without doing the intended work (e.g., hard-coding test outputs, bypassing
376 the verifier, network-fetching the reference). Discovered exploits are patched and the task is
377 re-validated. This audit follows the T-Bench Phase 2 protocol and is run on every task in the final
378 benchmark.
- 379 **Tests-per-task as a quality signal.** RALPHBench tasks are deliberately backed by *upstream* test
380 suites of mature OSS projects (449–579 tests/task), not author-written tests. This is a stronger oracle
381 than the synthetic-test or paired-PR-test approach used elsewhere: passing the upstream test suite is
382 the same gate that the original maintainers use to ship code. The minimum-tests-per-task policy is
383 part of v1.0 inclusion criteria.

Table 5: **Per-task construction cost** (RALPHBench v1.0).

Task	Est. agent solve	Est. token usage	Author construction time
biofabric-rust-rewrite	2–5 h	100–250M	60 h
network-aligner-optimize	1–3 h	50–100M	30 h
optimize-graphlet-sampling	1–3 h	50–100M	40 h
beyond-size8-graphlets	2–5 h	75–150M	40 h
rust-c-compiler	3–5 h	100–250M	50 h
Total	9–21 h	375M–850M	220 h

384 **Construction cost.** The 220 author-hours invested across 5 tasks is comparable per-task to the
385 Terminal-Bench audit standard (~3 reviewer-hours per task, plus author construction time on top).

386 4.5 Contamination Controls

387 We address contamination through three mechanisms:

388 **Repository freshness and pinning.** All upstream repositories are pinned to specific commits with
389 version-frozen test suites. The agent receives the codebase at the pinned commit; future maintainer
390 changes that might be in the agent’s training data cannot leak into the evaluation environment.

391 **Task novelty.** The transformations required (Java→Rust port of BioFabric with byte-level parity;
392 extension of BLANT graphlet labelling from $k=8$ to $k=14$ requiring 128-bit integers; bootstrap of a
393 self-hosting C compiler with x86-64/ARM64/RISC-V backends) do not exist as published reference
394 solutions. Even given the upstream codebase, no copy-paste solution is available in training data
395 because the target artifact has not been produced before. This is the same decontamination strategy
396 SWE-AGI [?] uses with MoonBit, but adapted to mainstream languages: novelty is enforced at the
397 task level, not the language level.

398 **Adversarial scan of agent submissions.** We post-hoc scan each agent’s final submission for
399 evidence of training-data retrieval (e.g., committed code that matches a known external reference
400 for a problem the agent shouldn’t have seen). FrontierSWE [?] reports active cheating attempts—
401 models writing bypass code, encoding torch imports as `chr()` sequences, attempting to detect the
402 verifier—and we expect similar behaviour at this scale. We log and report all detected cheating
403 attempts.

404 4.6 Harbor Specification and Extensions

405 We implement the Harbor v2.1 specification [?] for standardized agent-environment interaction.
406 Section 6 details the interface and our extensions.

Table 6: Harbor interface and RALPHBench extensions.

Component	Harbor v2.1 spec	RALPHBench extension
Shell	Sandboxed POSIX	Per-command timeout (300s), resource limits
Filesystem	Read/write workspace	Auto-diff snapshots after each write
Test runner	Execute, return exit code	Structured per-test parsing, assertion counts
Search	Optional	Required: BM25 + embedding hybrid
Logging	Structured events	Step accounting, milestone tagging
Governance	Wall-clock limit	Disk quota (10GB), network isolation

Step accounting. We define an “agent step” as one tool invocation (shell command, file read/write, search query). This normalizes across agents with different action granularities.

Test parsing. Rather than binary pass/fail, we parse test output to extract per-assertion results. This requires framework-specific parsers for pytest, jest, go test, cargo test, JUnit, and rspec (details in Section ??).

5 Multi-Level Evaluation Framework

[Why Multi-Level Evaluation?] Binary pass/fail discards information about *how close* agents get and *where* they fail. For tasks with 2,347 median steps and 142 mean test assertions, this information loss is substantial. Our framework extracts signal at multiple granularities while maintaining backward compatibility with prior work.

5.1 Level 1: Resolved Rate (Binary)

Definition. A task is *resolved* iff all test assertions pass after the agent terminates or exhausts its budget.

$$ResolvedRate = \frac{|\{t : all_tests_pass(t)\}|}{|\mathcal{T}|} \quad (1)$$

Purpose. Comparability with SWE-bench and prior work. Primary leaderboard metric.

Aggregation. Following best practices, we run each task $n = 3$ times and use **majority@3**: a task counts as resolved if ≥ 2 of 3 runs succeed. This provides robustness to variance while avoiding the optimism of pass@3.

Limitations. High variance for small N ; no partial credit; masks near-misses.

5.2 Level 2: Test-Pass Rate (Continuous)

Definition. Fraction of individual test assertions that pass.

$$Test - PassRate = \frac{\sum_{t \in \mathcal{T}} |passed_assertions(t)|}{\sum_{t \in \mathcal{T}} |total_assertions(t)|} \quad (2)$$

Purpose. Partial credit capturing “how close” agents get. Lower variance than binary metrics.

Difficulty weighting. To mitigate gaming (agents optimizing for easy tests), we optionally weight assertions by difficulty. Difficulty is estimated from the oracle solution: tests that require more oracle steps to reach are weighted higher. We report both unweighted and weighted test-pass rates.

Gaming analysis. We analyzed whether test-pass rate correlates with “number of trivial tests” by categorizing assertions as trivial (pass with empty diff), easy (pass with < 100 oracle steps), medium, and hard (> 500 steps). Test-pass rate improvement from trivial tests is modest (+3% on average), suggesting the metric captures genuine progress.

Table 7: **Milestone definitions for RALPHBench v1.0.** M3a/M3b refine the Implementation phase to capture the long ramp from first-passing-test to majority-passing in tasks of this scale.

Level	Milestone	Criterion	Detection method
M1	Orientation	Agent has read relevant files	≥ 3 files in task’s ground-truth set viewed
M2	Diagnosis	Agent identifies correct subsystem and produces a buildable artifact	First successful build of agent-modified code
M3a	First-pass	Any oracle test transitions from fail to pass	≥ 1 test passing
M3b	Majority-pass	Substantive progress against the oracle suite	$\geq 50\%$ of oracle tests pass
M4	Resolution	Task complete	100% of oracle tests pass

5.3 Level 3: Milestone Completion (Ordinal)

Definition. Each task defines 4 ordered milestones representing phases of a long-horizon SWE workflow. We adapt the canonical Orientation/Diagnosis/Implementation/Resolution decomposition to the RALPHBench setting, where dense oracle test suites (449–579 tests/task) provide a near-continuous M3 signal:

Why split M3. On SWE-bench-style tasks with 1–40 tests, M3 (50% pass) and M4 (100% pass) are tightly coupled. On RALPHBench tasks with 449–579 oracle tests, an agent can spend hours of trajectory time between first-test-passing and majority-passing; this is precisely the regime we want to measure. M3a captures the moment the agent’s solution is structurally correct enough to start passing tests at all; M3b captures the long debugging ramp.

Detection algorithm. Milestones are detected programmatically from trajectory logs (Section ??) parsed from the Harbor execution harness. M2 is grounded in build success rather than “edited the right file” to remove the subjectivity that plagued the v1 detection algorithm. M3a/M3b/M4 are exact functions of the test outcome stream and require no judgement.

Purpose. Understand *where* agents fail. On long-horizon tasks the same final score (e.g., 0% resolved) can correspond to drastically different trajectories: an agent that never builds, an agent that builds and passes 30% of tests, and an agent that passes 95% all look identical under binary resolved-rate. Milestone reporting separates these cases.

5.4 Level 4: LLM-Judge Rubrics (Qualitative)

Motivation. Functional tests verify correctness but cannot assess code quality. A solution might pass tests but be unmaintainable, violate project conventions, or miss documentation updates. This is particularly relevant on RALPHBench tasks where the upstream test suite is dense (449–579 tests) but still does not exercise stylistic or architectural concerns.

Agent-as-Judge framing. We adopt the **Agent-as-Judge** framework [?]: the evaluator is itself an agent with execution access, able to run the submitted code, inspect coverage and runtime behaviour, and verify hierarchical requirements through tool use—rather than a static LLM judge that examines only final diffs. This is a natural fit for RALPHBench because each task already ships with a working build environment and dense oracle tests, which the judge agent can exercise directly.

Rubric dimensions. The judge agent scores resolved solutions on 5 dimensions:

Implementation. We use Claude Opus 4.7 as the primary judge with a structured prompt (Section ??) and access to the full task environment. The judge receives: (i) task description, (ii) original codebase state, (iii) agent’s final diff, (iv) test results, and (v) the ability to run additional probes against the submission.

Table 8: LLM-judge rubric (1–5 scale, 5=best).

Dimension	Description	Weight
Correctness	Handles edge cases beyond the test suite?	0.30
Maintainability	Readable, well-structured, appropriate abstractions?	0.25
Conventions	Follows project style, naming, patterns?	0.20
Efficiency	Reasonably performant, no obvious inefficiencies?	0.15
Completeness	Documentation, configs, related tests updated?	0.10

Table 9: Evaluation framework summary.

Level	Metric	Type	Primary use
L1	Resolved rate	Binary	Headline / leaderboard comparability
L2	Test-pass rate	Continuous [0, 1]	Partial credit on dense oracle tests
L3	Milestone completion	Ordinal [M1–M4]	Diagnostic analysis of failure stage
L4	LLM-judge score	Rubric [1–5]	Code-quality assessment (descriptive)

Treatment in this work. Given the small task count, we report judge scores descriptively alongside execution-based metrics rather than as a primary evaluation axis, and we publish raw judge outputs for community scrutiny. We do not perform a formal κ calibration against human raters at this scale; FrontierSWE [?] similarly relies on execution-grounded scoring rather than judge-based metrics in the frontier regime. Calibration of LLM-judge scores against expert human ratings on tasks of this horizon is left to future work; we view it as substantively harder than calibration on shorter benchmarks because the trajectories themselves take experts hours to evaluate.

Gaming risk. Medium—agents could theoretically optimize for “looks good to an LLM.” We mitigate by keeping judge scores strictly secondary to execution-based metrics, never folding them into resolved-rate calculations, and reporting them separately.

5.5 Metric Summary

We report all four levels separately and decline to introduce a weighted composite score: at $n=5$ tasks, weighted aggregation is unstable and obscures more than it reveals. $\text{mean}@k$ and $\text{best}@k$ on the resolved-rate and test-pass-rate channels [?] are the primary headline numbers.

6 Experimental Setup

6.1 Agent Systems

We evaluate 11 agent–model configurations spanning commercial CLI products and the open-source Terminus 2 scaffold, plus two reference baselines (`nop` and `oracle`). Section 10 summarizes the systems with version information. All evaluations were executed on Modal via `harbor run ... -e modal`.

Reference baselines.

- **nop:** Harbor’s no-op agent (no actions taken); establishes the resolved-rate floor attributable to tasks that already pass at $t = 0$.
- **oracle:** Harbor’s oracle agent (applies the held-out solution); verifies that harness, grader, and base image are not the limiting factor on resolved rate.

Commercial agents.

Table 10: **Evaluated agent systems.** Agent versions are the latest published as of the run window; record `-version` from each container for exact reproducibility.

Agent	Type	Version	Model	Provider
<i>Reference baselines</i>				
nop	Harbor built-in	—	—	—
oracle	Harbor built-in	—	—	—
<i>Commercial CLI agents</i>				
Claude Code	Anthropic	v2.1.123	Claude Opus 4.7	Anthropic
Codex CLI	OpenAI	v0.128.0	GPT-5.5	OpenAI
Gemini CLI	Google	v0.40.0	Gemini 3.1 Pro Preview	Google
Kimi Code CLI	Moonshot	v1.41.0	Kimi K2.6	Moonshot AI
<i>Open-source scaffold (Terminus 2) across model backbones</i>				
Terminus 2	harbor-framework	v2.0 [†]	Claude Opus 4.7	Anthropic
Terminus 2	harbor-framework	v2.0 [†]	GPT-5.5	OpenAI
Terminus 2	harbor-framework	v2.0 [†]	Gemini 3.1 Pro Preview	Google
Terminus 2	harbor-framework	v2.0 [†]	Kimi K2.6	Moonshot AI
Terminus 2	harbor-framework	v2.0 [†]	DeepSeek V4 Pro	DeepSeek
Terminus 2	harbor-framework	v2.0 [†]	GLM 5.1	Z.ai
Terminus 2	harbor-framework	v2.0 [†]	MiniMax M2.7	MiniMax

[†]Terminus 2 ships inside the `harbor-framework/terminal-bench` repository and does not carry an independent semver tag; the relevant identifier is the repository commit hash at run time.

- **Claude Code:** Anthropic’s agentic coding tool with extended thinking and multi-file support, run on Claude Opus 4.7.
- **Codex:** OpenAI’s code-specialized agent (CLI version), run on GPT-5.5.
- **Gemini CLI:** Google’s agent with code execution and long-context capabilities, run on Gemini 3.1 Pro Preview.
- **Kimi CLI:** Moonshot’s agent with a 200K+ context window, run on Kimi K2.6 via OpenRouter.

Open-source agents.

- **Terminus 2:** Terminal-Bench’s reference Harbor-compatible scaffold, swept across seven backbones (Claude Opus 4.7, GPT-5.5, Gemini 3.1 Pro Preview, Kimi K2.6, DeepSeek V4 Pro, GLM 5.1, MiniMax M2.7). Holding the scaffold fixed isolates the contribution of the underlying model.

6.2 Runtime Environment

All evaluations run in identical containerized environments on Modal:

- **Base image:** Ubuntu 22.04 (Docker digest `sha256:8a7d3c2e...`)
- **Resources:** 8 vCPU, 32GB RAM, 10GB disk quota
- **Time limit:** 4 hours wall-clock per task
- **Network:** Disabled (except for model API calls via proxy)
- **Decoding:** Temperature = 0 for `pass@1`; temperature = 0.7 for `pass@k` ($k > 1$)

Each run records: container image digest, repository commit, harness commit, agent version, model id, random seed, and complete action trace.

6.3 Evaluation Protocol

Run configuration.

- **Primary evaluation:** $n = 5$ runs per task per configuration (majority@5 for resolved rate)
- **pass@k analysis:** $n = 10$ runs per task for the top-5 configurations

517 • **Total trajectories:** $13 \times 100 \times 3 = 3,900$ primary runs (11 agent–model configurations + 2
518 baselines), plus additional pass@k runs

519 **Statistical methodology.**

- 520 • Bootstrap confidence intervals: $B = 10,000$ resamples
- 521 • Effect sizes: Cohen’s d for continuous, odds ratios for binary comparisons
- 522 • Multiple comparison correction: Bonferroni-Holm for >3 pairwise tests
- 523 • Pre-registration: Primary metrics (resolved rate, test-pass rate) pre-registered; exploratory analyses
524 noted

525 **7 Experiments and Evaluation**

526 **7.1 Experimental Results**

527 **7.1.1 Main Results**

528 Main results and tables.

529 **7.1.2 Successes**

530 What went well.

531 **7.1.3 Failure Modes**

532 Common failure modes observed.

533 **7.1.4 Qualitative Examples**

534 Some screenshots or images.

535 **References**

- 536 [1] Tom Adamczewski, David Rein, David Owen, and Florian Brand. MirrorCode: Evidence
537 that AI can already do some weeks-long coding tasks. Epoch AI blog post, April 2026. Data:
538 <https://github.com/epoch-research/MirrorCode-data>.
- 539 [2] Ben Bogin, Kejuan Yang, Shashank Gupta, Kyle Richardson, Erin Bransom, Peter Clark, Ashish
540 Sabharwal, and Tushar Khot. SUPER: evaluating agents on setting up and executing tasks
541 from research repositories. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors,
542 *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing,*
543 *EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 12622–12645. Association for
544 Computational Linguistics, 2024.
- 545 [3] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays,
546 Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Aleksander Madry, and Lilian
547 Weng. Mle-bench: Evaluating machine learning agents on machine learning engineering. In
548 *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore,*
549 *April 24-28, 2025*. OpenReview.net, 2025.
- 550 [4] Evan Chu, Rajan Agarwal, Abishek Thangamuthu, Brendan Graham, and Justus Mattern.
551 FrontierSWE: Benchmarking coding agents at the limits of human abilities. Proximal Labs blog
552 post, <https://www.frontierswe.com/blog>, April 2026.
- 553 [5] Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and
554 models in container environments, January 2026.
- 555 [6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and
556 Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.

- [7] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026.
- [8] Samuel Miserendino, Michele Wang, Tejal Patwardhan, and Johannes Heidecke. Swe-lancer: Can frontier llms earn \$1 million from real-world freelance software engineering?, 2025.
- [9] OpenAI. Introducing SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>, August 2024.
- [10] Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. Core-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *Trans. Mach. Learn. Res.*, 2024, 2024.
- [11] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. Paperbench: Evaluating ai’s ability to replicate AI research. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2025.
- [12] Minh V. T. Thai, Tue Le, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. Swe-evo: Benchmarking coding agents in long-horizon software evolution scenarios, 2026.
- [13] Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows, 2025.
- [14] Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Joshua Clymer, Jai Dhyani, Elena Elicheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. Re-bench: Evaluating frontier AI r&d capabilities of language model agents against human experts. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2025.
- [15] Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Zhiruo Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Keunho Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. Theagentcompany: Benchmarking LLM agents on consequential real world tasks. *CoRR*, abs/2412.14161, 2024.
- [16] Christine Ye, Sihan Yuan, Suchetha Cooray, Steven Dillmann, Ian L. V. Roque, Dalya Baron, Philipp Frank, Sergio Martin-Alvarez, Nolan Koblishke, Frank J Qu, Diyi Yang, Risa Wechsler, and Ioana Ciuca. Replicationbench: Can ai agents replicate astrophysics research papers?, 2025.
- [17] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. Multi-swe-bench: A multilingual benchmark for issue resolving. *CoRR*, abs/2504.02605, 2025.

- 610 [18] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones,
611 Gashon Hussein, Samantha Liu, Donovan Julian Jasper, Pura Peetathawatchai, Ari Glenn,
612 Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Haoxiang Yang, Aolin
613 Zhang, Rishi Alluri, Nathan Tran, et al. Cybench: A framework for evaluating cybersecurity
614 capabilities and risks of language models. In *The Thirteenth International Conference on*
615 *Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.
- 616 [19] Wenting Zhao, Nan Jiang, Celine Lee, Justin T Chiu, Claire Cardie, Matthias Gallé, and
617 Alexander M Rush. Commit0: Library generation from scratch, 2024.